

Whisper Local Optimization for Custom Context

Version: 1.0

Target System: Whisper Local (Python/C++ Implementation)

User: Isaiah N Washington (Developer / Audio Engineer / Gamer)

1. EXECUTIVE SUMMARY

The goal of this project is to optimize a local Whisper transcription pipeline to accurately recognize domain-specific terminology, handle complex syntax (code vs. prose), and resolve homophone ambiguities based on the specific profile of the user. The system must prioritize accuracy for **Audio Engineering**, **Software Development (Python/AI)**, and **Gaming** contexts.

2. USER PROFILE & VOICE CONTEXT

- **Speaker Identity:** Isaiah N Washington.
- **Speaking Style:** Moderate pace (~130-150 wpm), frequently switches between casual conversational English and highly technical jargon.
- **Key Environment:** The system runs locally on hardware utilizing CUDA cores.
- **Primary Difficulty:** The model currently fails to distinguish between homophones ("write" vs "right") and incorrectly formats technical strings (IP addresses, API keys, snake_case variables).

3. DOMAIN-SPECIFIC VOCABULARY (HOTWORDS)

*The coding assistant must ensure the following terms are prioritized in the model's vocabulary or **initial_prompt** biasing.*

A. Audio Engineering & Music Production

- **Hardware:** Shure SM7B, Audient iD14, XLR, Preamp, Phantom Power (48V).
- **Software/DAW:** Ableton Live, Pro Tools, VST plugins.
- **Technical Terms:** Sidechain compression, High-pass filter (HPF), Low-pass filter (LPF), Q-factor, THD+N (Total Harmonic Distortion + Noise), Signal-to-Noise Ratio (SNR), Hz (Hertz), kHz (Kilohertz), Bit-depth.

B. Software Development & AI

- **Languages/Formats:** Python, JSON, C++, CUDA, WSL (Windows Subsystem for Linux), GitHub.
- **Libraries:** **pandas** (as pd), **numpy** (as np), PyTorch, TensorFlow, Transformer models.

- **Syntax (Spoken vs. Written):**
 - "Underscore" $\rightarrow$ `_`
 - "Camel Case" $\rightarrow$ `camelCase`
 - "Snake Case" $\rightarrow$ `snake_case`
 - "Init" $\rightarrow$ `__init__` (when in Python context)

C. Gaming

- **Titles:** Dota 2, RuneScape.
- **Jargon:** "Mid lane," "Gank," "DPS," "Aggro."

4. FORMATTING & LOGIC RULES

The coding assistant must implement post-processing logic or prompt engineering to strictly enforce these rules:

Rule 1: The "Grandma" Safety Check (Punctuation)

- **Requirement:** The system must strictly apply the Oxford comma and dependent clause commas to prevent meaning alteration.
- **Failure Example:** "Let's eat grandma."
- **Success Target:** "Let's eat, Grandma."

Rule 2: IP Address & Port Standardization

- **Requirement:** Numerical sequences following the pattern of IPv4 must **never** contain spaces between octets.
- **Failure Example:** "127. 0. 0. 1 : 8080"
- **Success Target:** "127.0.0.1:8080"

Rule 3: Homophone Disambiguation via Context

- **Logic:**
 - If context implies *direction* or *correctness* $\rightarrow$ "Right"
 - If context implies *creation/input* $\rightarrow$ "Write"
 - If context implies *location* $\rightarrow$ "There"
 - If context implies *possession* $\rightarrow$ "Their"
 - If context implies *identity/state* $\rightarrow$ "They're"

Rule 4: Code Block Detection

- **Trigger:** If the user speaks keywords like "def," "function," "import," or "variable," the output should shift to code-friendly formatting (monospaced or raw text representation) rather than phonetic spelling.

5. VALIDATION DATA (GROUND TRUTH)

Use this dataset to test the success of the implementation.

Test Case A: The Stress Test

Input Audio: "My local host address is 127. 0. 0. 1: 8080. If I say sudo apt-get update, does it hyphenate apt-get?"

Required Output: "My localhost address is 127.0.0.1:8080. If I say `sudo apt-get update`, does it hyphenate `apt-get`?"

Test Case B: Audio Jargon

Input Audio: "We are checking for the THD plus N rating and the signal to noise ratio."

Required Output: "We are checking for the THD+N rating and the Signal-to-Noise Ratio."

6. IMPLEMENTATION DIRECTIVES FOR AI AGENT

To the Coding Agent reading this:

1. **Modify the `initial_prompt`:** Inject the vocabulary list from Section 3 into the Whisper configuration `initial_prompt` string. This is the most effective way to bias the model.
2. **Regex Post-Processing:** Create a Python script using standard Regex (`re` library) to clean up IP addresses (Rule 2) and force-correct common homophone errors (Rule 3) based on n-gram context.
3. **Parameter Tuning:** Ensure `condition_on_previous_text` is set to `True` to maintain context across long pauses.